Bringing it all together!

SyoSil ApS © 2026

Content

- Full course quick recap
- CRV Best Practices
  - And other things ☺

SyoSil ApS © 2026

Full Course Quick Recap

SyoSil ApS © 2026

What Have We Learned So Far?

- CRV with UVM (PyUVM and cocoTB)
  - Lesson 00 to 06
- Top level instantiation if DUT in cocoTB/PyUVM framework
  - Lesson 07
- PyUVM Tests, Env & Vseqs
  - Lesson 08
- UVC Integration
  - Lesson 09
- Stimuli randomization
  - Lesson 10
- Coverage
  - Lesson 11
- Checkers
  - Lesson 12
- Reference Models
  - Lesson 12/13
- Verification Plan
  - Lesson 14

- Lets redo it!
  - New DUT
  - Reuse TB from SAT

SyoSil ApS © 2026

New DUT/RTL Design

SyoSil ApS © 2026

New DUT

- Simple summarizer
- Accepts inputs and adds the new and previous input
- Uses the sSDT protocol

```
module summer #(parameter DATA_W = 8  // Data signal bit size) (
  input   logic               clk,
  input   logic               rst,
  input   logic               in_valid,   // input data valid
  input   logic [DATA_W-1:0]  in_data,    // input data
  output  logic               out_valid,  // output data valid
  output  logic [DATA_W-1:0]  out_data    // output data
);

  reg [DATA_W-1:0] prev_val;

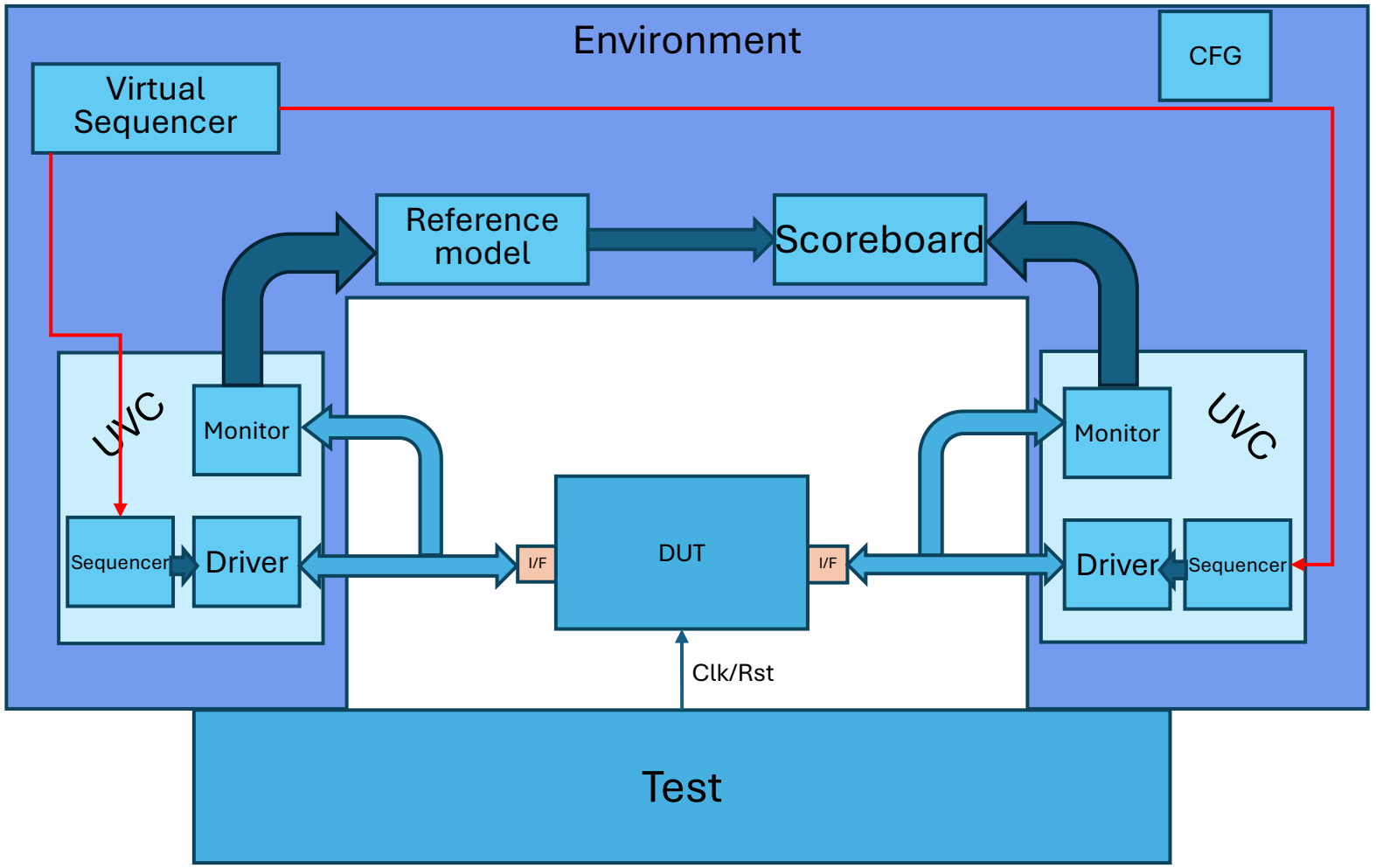
  // Main loop
  always_ff @(posedge clk, posedge rst) begin
    if (rst === 1'b0) begin
      out_valid <= 'b0;
      out_data <= 'b0;
      prev_val <= 'b0;
    end else begin
      if (in_valid === 1'b1) begin
        out_valid <= 'b1;
        out_data <= in_data + prev_val;
        prev_val <= in_data;
      end else begin
        out_valid <= 'b0;
        out_data <= 'b0;
      end
    end
  end
endmodule: summer
```

SyoSil ApS © 2026

Reused Test Bench

SyoSil ApS © 2026

UVM Test Bench – Generic Structure



SAT TB vs Summer TB – Comparison

- After normalizing SAT/Summer names, the current major differences are:
  - Summer uses active-low reset; SAT uses active-high reset – Can be ignored.
  - Reference models – Expected
  - SAT has design checkers
- The following remain essentially equivalent
  - Testbench configuration
  - Producer and consumer UVC agents
  - Virtual sequencer structure
  - Scoreboard comparison logic
  - Default and random test factory overrides
  - Coverage-report generation

- AI Question
  - What is your estimate for a percentage of code reuse between the two TBs?
- AI answer
  - Estimated reuse
    - Core TB architecture and shared files: approximately 80%
    - Entire SAT TB suite, including omitted tests/sequences/checkers: approximately 60–65%
    - Overall practical estimate: about 70% code reuse
  - Most changes are DUT-specific behavior: reset polarity, reference model, coverage, checkers

SyoSil ApS © 2026

CRV Best Practices
- And other things!

SyoSil ApS © 2026

Contents

- General
- Python and PyUVM
- Regression & Flow
- Closing Coverage
- Reset, X Propagation, Gatelevel Simulations

SyoSil ApS © 2026

General

SyoSil ApS © 2026

General - Strategy

- Strategy
  - Make a verification strategy for
    - Module level verification
    - Subsystem level verification
    - Chip/System level verification
  - Introduce maturity levels for the TBs, So a TB can be in e.g. A, B or C state
  - Keep track of all TBs and the current state

- Coding Guidelines for TBs
  - Naming conventions
    - Python packages
    - Python modules
    - TB components
    - Class names
    - Variable names
    - …
  - Good for new employees

- Recommended workshops
  - General strategy
  - Stimuli
  - Coverage
  - Testbench architecture
  - Coverage Closure

SyoSil ApS © 2026

General – Module Level Verification

- Start by identifying
  - Stimuli
  - Checkers (Assertions/Reference Models)
  - Coverage
- Through e.g. workshops!

- And ensure it is independent of each other

SyoSil ApS © 2026

General – Test Bench

- Developement Steps
  - TB Skeleton
  - UVCs
    - Reuse?
  - Reference Model?
    - Have to implement one?
  - Checkers
  - Coverage Model

- Reuse, Reuse, Reuse
  - Think in reusable verification components
    - UVCs
    - Scoreboard
    - Stimuli classes
    - Checkers
      - Assertions
      - Performance
    - General coverage groups
- Register Model → Generate!

SyoSil ApS © 2026

General - Tools

- **Continous Integration** → Jenkins
- **Code Reviews** → Gerrit
- **Testbench Documentation** → Mark Up Language
  - Latex, AsciiDoc, MarkDown, RST, …
  - Doxygen/Natural Docs which can read the SV code as well

SyoSil ApS © 2026

Python and PyUVM

SyoSil ApS © 2026

Python

- Encapsulate reusable code, utilize
  - Python modules
  - Classes
- Parameterized Test Bench
  - PyUVM
    - Does not really have parameters
    - Store parameter values in configuration class for environment

SyoSil ApS © 2026

PyUVM

- Config DB
  - Only use the configuration database for **three** (one for PyUVM) thing(s)
  - **Sharing component configuration classes** between components
    - `ConfigDB().set(self, “env", "cfg", self.cfg)`
  - **Sharing virtual interfaces** (SV Only)
  - **Passing command line arguments** (SV Only)
    - PyUVM does not supports UVM command line args at the moment. Use argParse or other modules
    - Once, command line argument is passed, then store in ConfigDB or the configuration class (preferred)

SyoSil ApS © 2026

PyUVM

- Stimuli models
  - Always base class implementation without any legality constraints
  - Shared across testbenches if possible
  - For randomization of registers (Can be generated, see DVCon EU 2019 Paper)

- Coverage
  - Central Collector
    - TLM1/2 sockets for hooks
    - Ensures future crosses can be implemented
  - Make cover groups configurable, especially if it is in shared code like a UVC
    - Configurable is preferred over coverage exclusions
  - Use functional coverage, assertion coverage and structural coverage (line, cond, toggle and branch)
  - Always validate with known data
  - Check for false positives

SyoSil ApS © 2026

PyUVM

- SCB
  - Architectural separation
  - Always do inverse testing (Invalidate RTL/REF model or stimuli) and check that the SCB fails

- File structure for a normal testbench
  - **`tb.sv/top.sv`:** Top level of the testbench (Verilog part)
  - **`<testbench name>_common.py`:** Macros, Type definitions, Parameters, etc.
  - **`<testbench name>_tb.py`:** Contains typically, env configuration, sequencer and env
  - **`<testbench name>_test.py`:** Base test and all other tests
  - **`<testbench name>_seqs.py`:** Base sequence and all other sequences
  - **`<testbench name>_sti.py`:** Stimuli classes
  - **`<testbench name>_cov.py`:** Coverage collector and cover groups etc.

SyoSil ApS © 2026

Regression
&
Flow

SyoSil ApS © 2026

Regression

- Must be well defined and checked into a control version system
- Can be based on random seeds or a set of fixed seeds.
    - Random seeds: Might require longer runtimes as more seeds will be needed.
    - Fixed seeds: Is more efficient when one later must rerun the regression (Can be generated based on test grading)

SyoSil ApS © 2026

Flow

- Support file dependencies for faster compilation
- Capable of run same UVM test in many configurations
  - CMD line args
  - Seeds (Both fixed and/or a set of random seeds)
  - Parameter sets
- Coverage merge across all tests both old and new → harvesting
- Trending (Coverage improvement over time)
- Utilize compute clusters

SyoSil ApS © 2026

Closing Coverage

SyoSil ApS © 2026

Closing Coverage

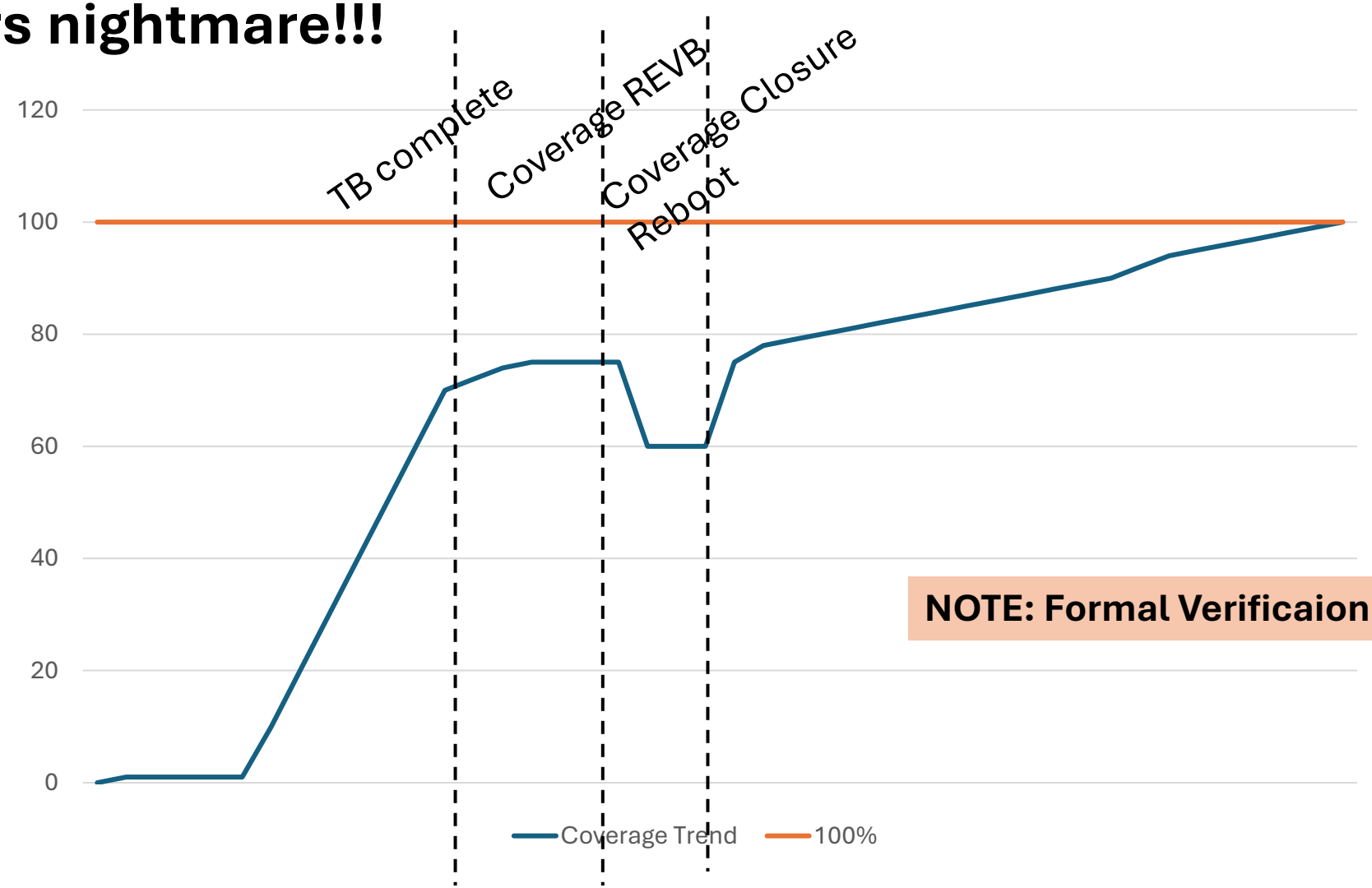
- Formal Unreachability Analysis
  - Provides excludes for structural coverage which cannot be reached → Either change RTL or do not spend time trying to close coverage for unreachable line, toggle etc.
  - Requires a formal tool (So far only commercially available)

- Closing techniques
  1. Run more random seeds (Harvesting) and see if coverage increases. If not, then random templates are saturated → (2)
  2. Implement new templates which is constrained differently
  3. Relax coverage criteria: Remove number of interesting cases in e.g. crosses
  4. Use formal to prove reachability/unreachability (You might be chasing ghosts)
  5. Exclusions (Split into individual files so it can be tracked if an exclusion is present because it is a general exclusion, project wide exclusion, or a specific DUT related exclusions for this project)

SyoSil ApS © 2026

Coverage Closing Curve vs Verification Complete

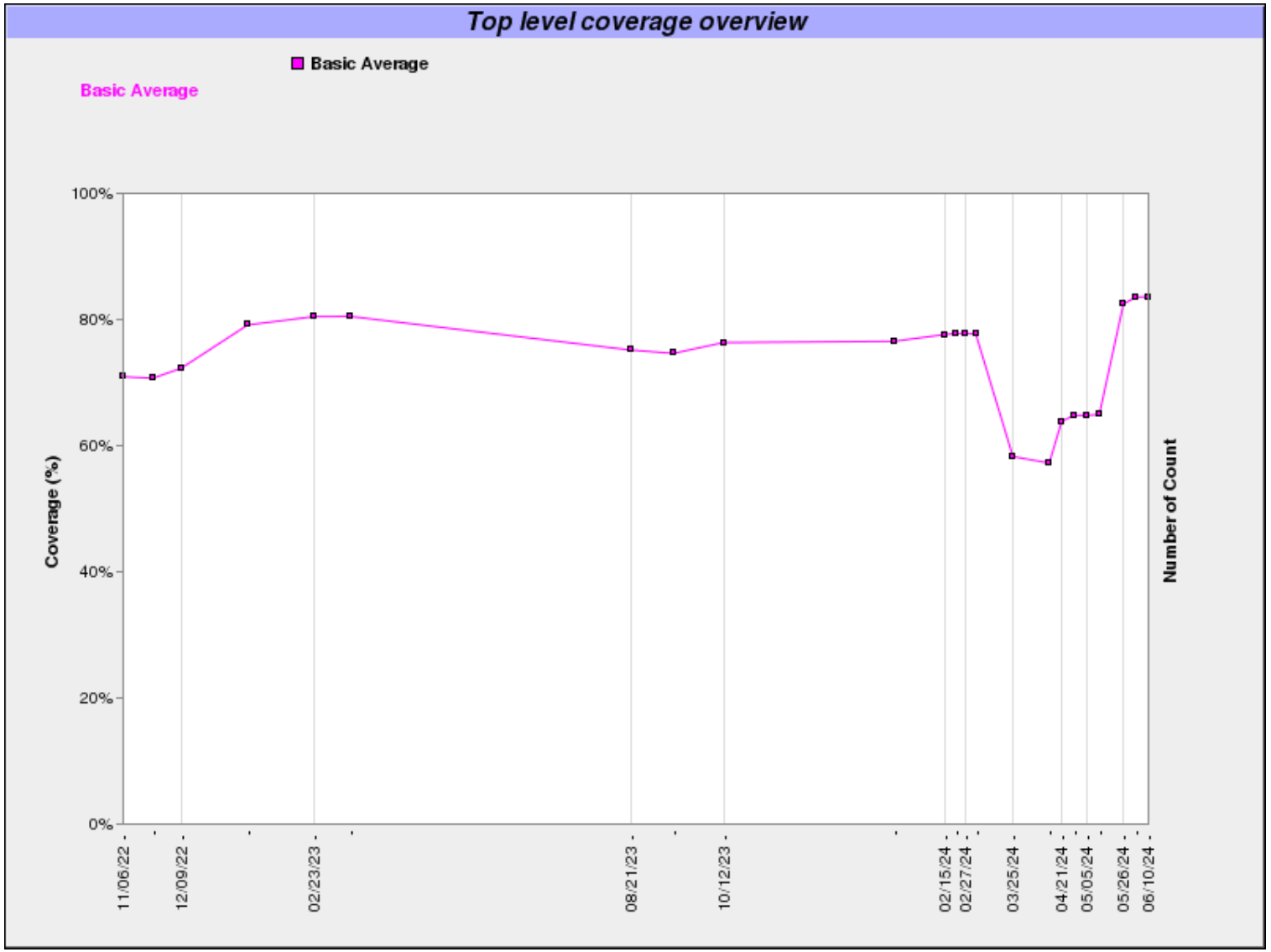
- **A mangers nightmare!!!**



NOTE: Formal Verificaion is even worse!!!!

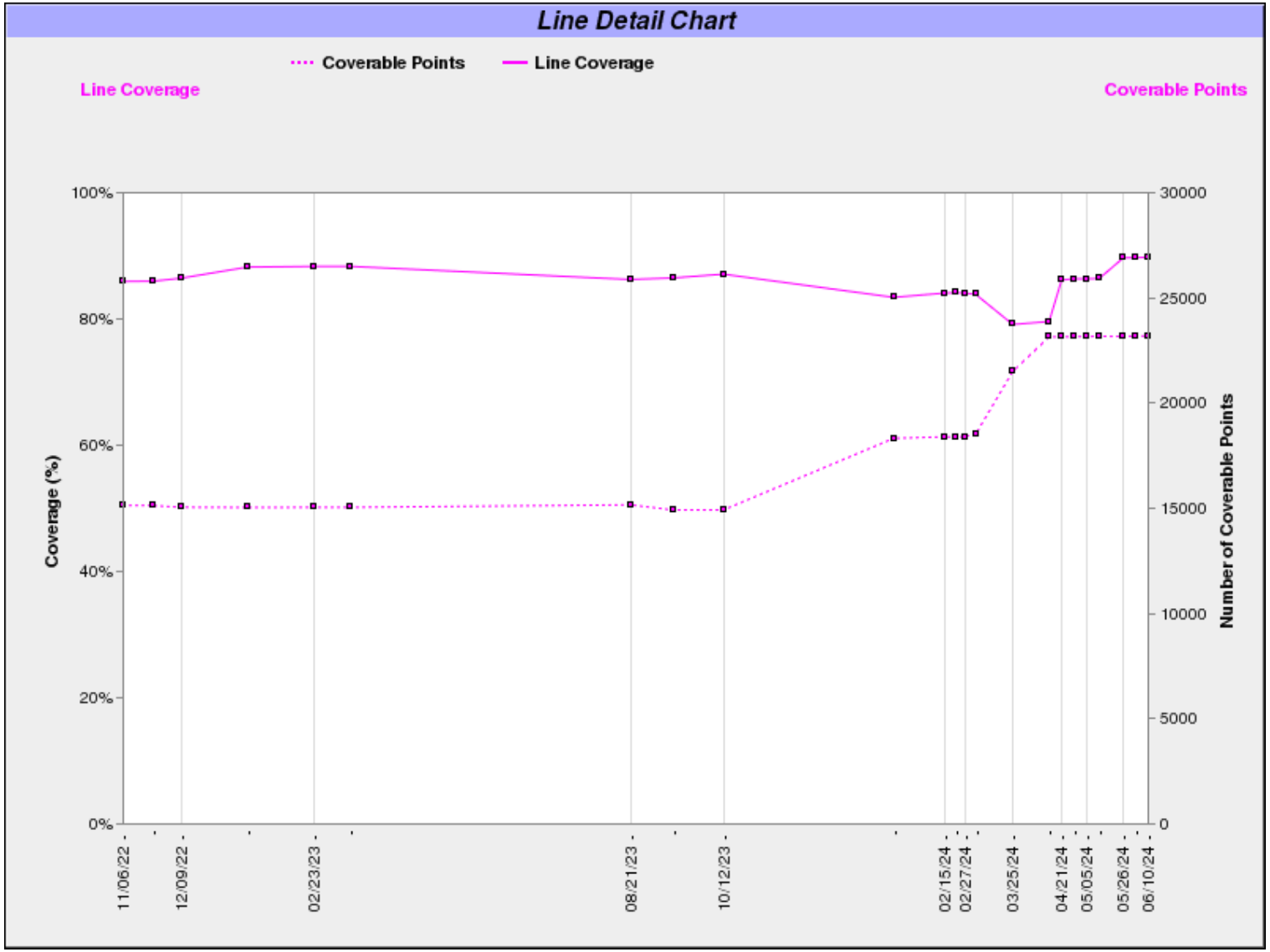
SyoSil ApS © 2026

Real Coverage Trend Example – Total Coverage



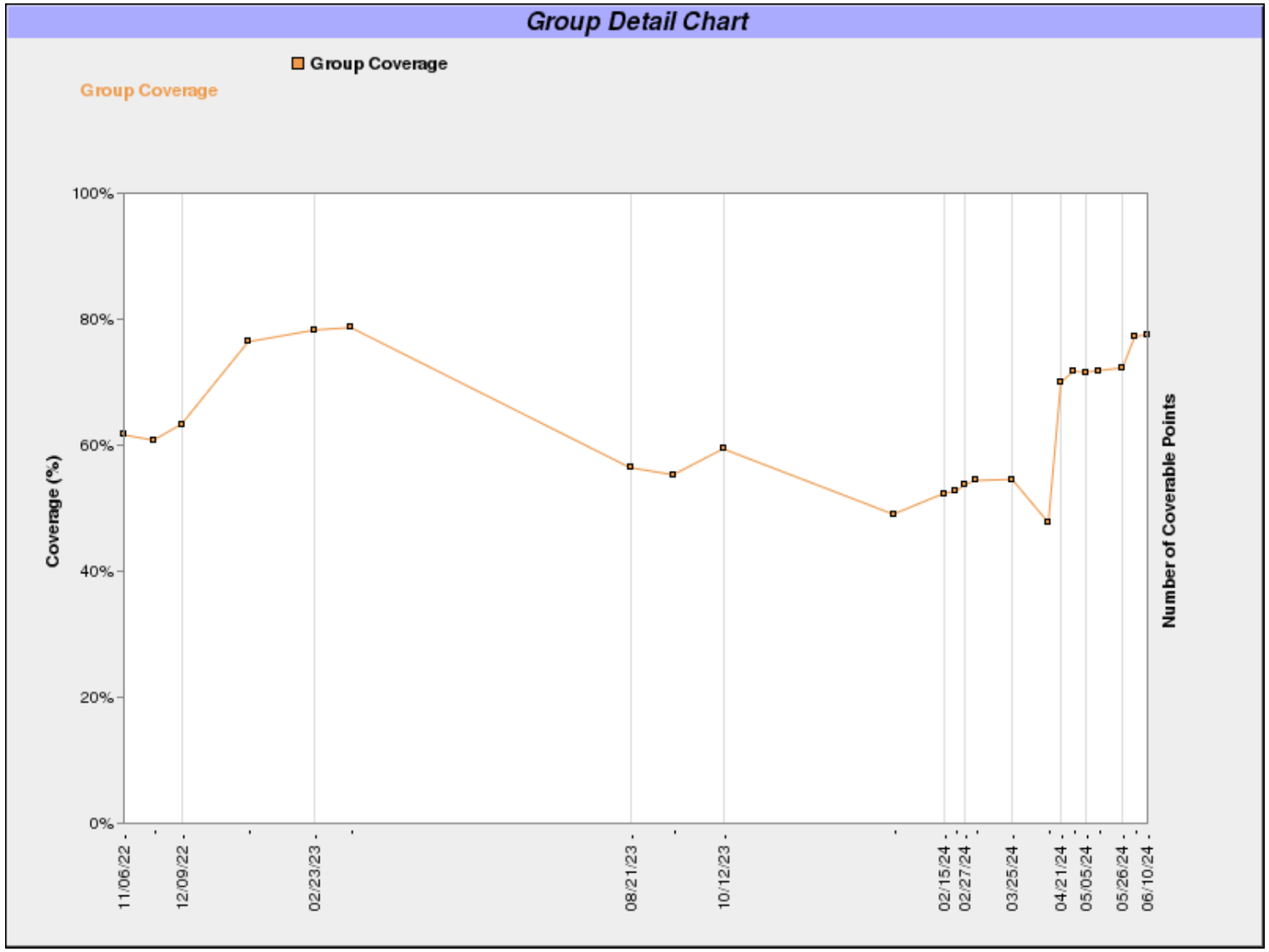
SyoSil ApS © 2026

Real Coverage Trend Example – Line Coverage



SyoSil ApS © 2026

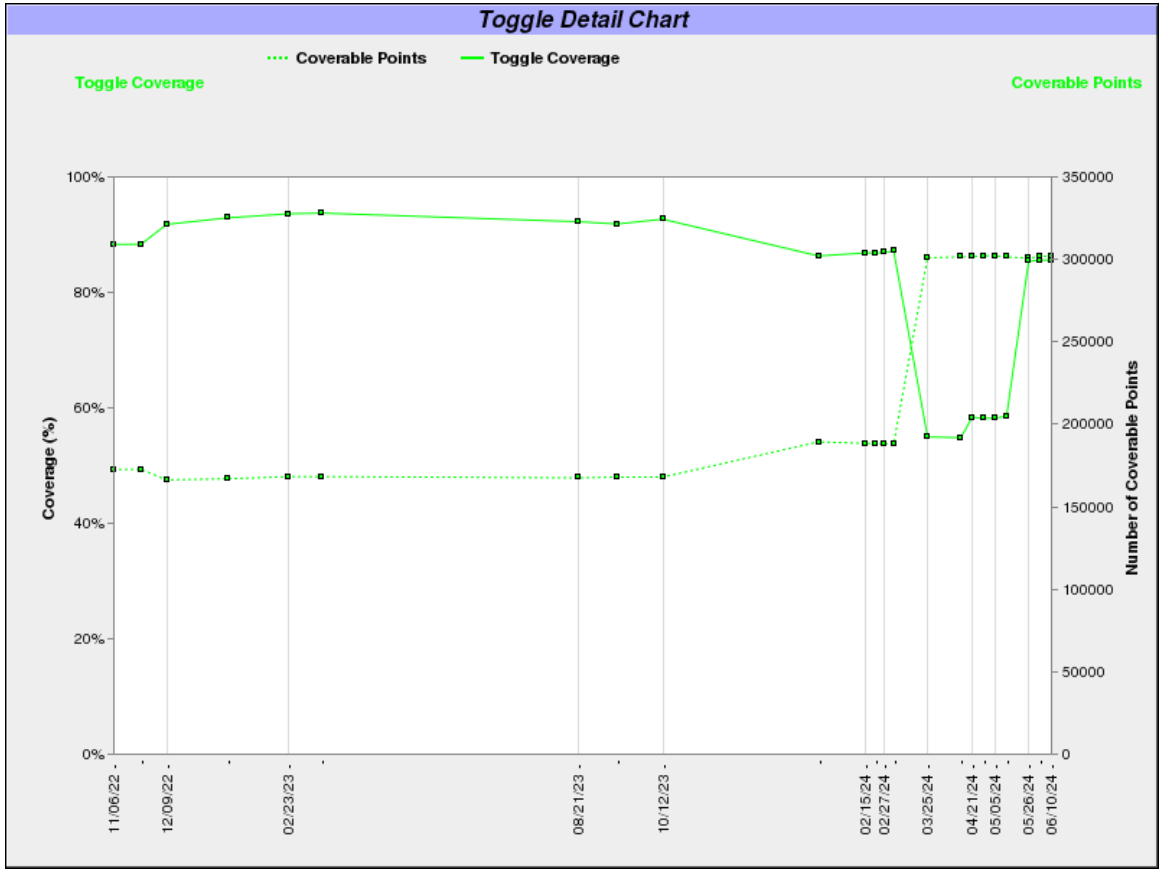
Real Coverage Trend Example – Functional Coverage

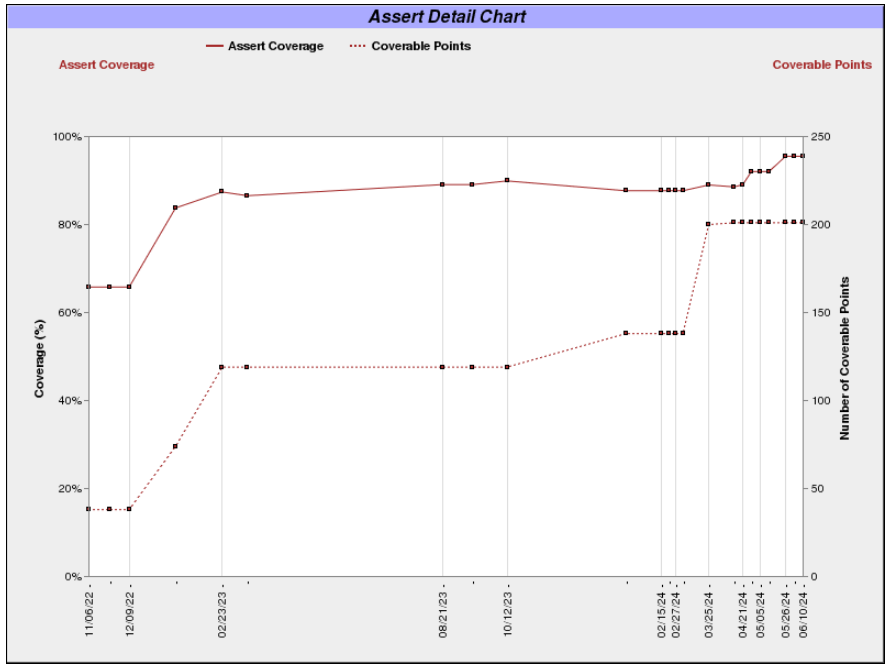


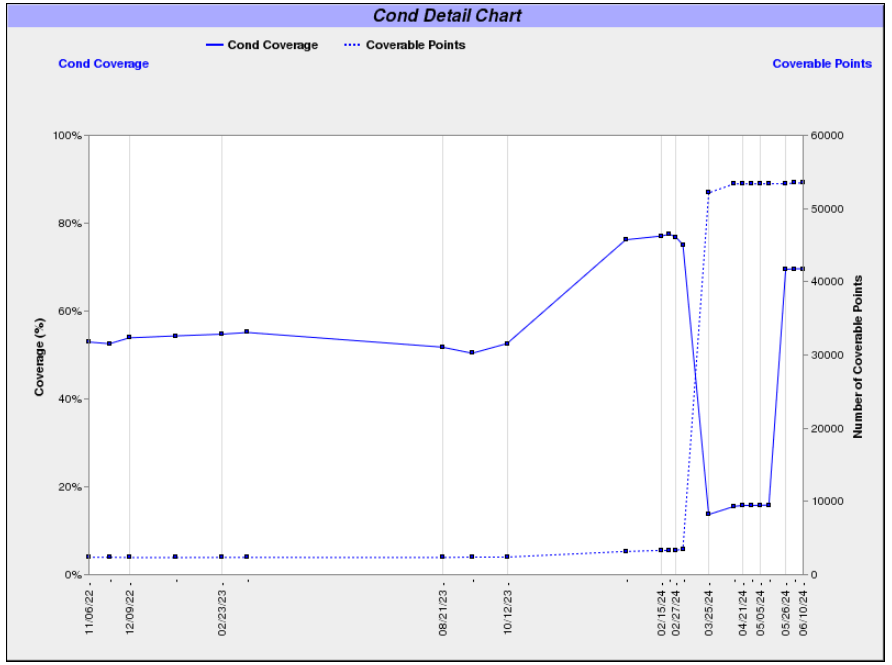
SyoSil ApS © 2026

Coverage Trend Example

Toggle, Assert and Conditional Coverage







SyoSil ApS © 2026

Reset, X Propagation & Gatelevel Simulations

SyoSil ApS © 2026

Reset, X Propagation, Gatelevel Simulations

- Reset
  - In general, only done once per simulation when X propagation is available
  - Shall be done if multiple asynchronous resets are inputs
- X-propagation
  - Can remove the need for gate level simulations at least at module level
  - Potentially requires individual licenses
- Gate level simulations
  - Can almost be avoided with careful planning (e.g. not at module level, only subsystem/system level)
  - Determine early if block needs GL simulations → Prevents/Makes it difficult to reuse white box probes/checks with GL simulations in module level TBs

SyoSil ApS © 2026

Anything Missing?

- Is there anything missing for the SAT verification?
  - What about the requirements → Check them out!
  - E.g.
    - Input valid and data propagation → Add checker
    - Overflow signal behaving correctly → Add checker
  - Exercise E13 helps you add these checkers

SyoSil ApS © 2026